Projeto: Desenvolvimento de um Relógio Digital de Pulso Utilizando Temporizadores

Universidade Federal do Rio Grande do Norte
Departamento de Engenharia de Computação e Automação
Sistemas Embarcados - 2026.2

1 Introdução

Temporizadores são recursos fundamentais em sistemas embarcados, pois permitem executar tarefas em intervalos de tempo determinados, controlar eventos periódicos e medir a duração de acontecimentos. Esses recursos estão presentes em diversas aplicações, como sistemas de automação, instrumentos eletrônicos, equipamentos médicos e dispositivos de uso pessoal. Um exemplo cotidiano de aplicação é o relógio digital de pulso. Esse dispositivo precisa manter uma contagem contínua de horas, minutos e segundos, atualizar seu visor periodicamente e permitir que o usuário configure a hora e utilize funcionalidades adicionais, como cronômetro, temporizador regressivo e alarme.

Neste projeto, você deverá projetar e programar, no Tinkercad, um sistema embarcado que simule o funcionamento de um relógio digital de pulso utilizando um Arduino Uno. O sistema deverá utilizar uma base de tempo para realizar a contagem dos segundos e atualizar as informações apresentadas em um display LCD. Além disso, deverá interpretar diferentes formas de acionamento dos botões, distinguindo pressionamentos rápidos de pressionamentos prolongados.

O principal desafio consiste em desenvolver uma lógica de temporização precisa, sem depender de atrasos bloqueantes excessivos, permitindo que o sistema execute simultaneamente a contagem do tempo, a leitura dos botões e a atualização do display. Como desafio adicional não obrigatório, o projeto poderá ser expandido para simular as principais funcionalidades de um relógio digital de pulso comercial, incluindo cronômetro, temporizador regressivo, alarme e iluminação do visor.

2 Especificações de Hardware

O circuito deve ser composto pelos seguintes elementos no simulador:

- 1 Microcontrolador Arduino Uno: Responsável pelo processamento das informações, controle da temporização, leitura dos botões e atualização do display.
- 1 Display LCD 16x2: Responsável por apresentar as informações do relógio, incluindo horas, minutos e segundos.
- 2 Botões: Serão utilizados para controlar as funcionalidades básicas do relógio:

  - Botão de Ajuste: permite entrar ou sair do modo de configuração e selecionar o campo que será ajustado.
  - Botão de Avanço: permite incrementar os valores de horas, minutos ou segundos durante a configuração.

- Resistores para os botões, caso necessário: Poderão ser utilizados resistores de pull-down ou pull-up para garantir níveis lógicos estáveis nas entradas digitais. Alternativamente, poderá ser utilizada a configuração `INPUT_PULLUP` do Arduino.

- Protoboard e fios de conexão: Utilizados para realizar as conexões elétricas entre os componentes.

3 Modelagem do Sistema

3.1 Contagem e Base de Tempo

O relógio deverá utilizar uma base de tempo para realizar a contagem dos segundos de maneira contínua. Para isso, poderá ser utilizada a função `millis()` do Arduino, que retorna o tempo decorrido desde a inicialização do microcontrolador em milissegundos. A cada intervalo de 1000 milissegundos, o sistema deverá incrementar a variável responsável pela contagem dos segundos.

A atualização deverá ser realizada sem utilizar a função `delay()`, evitando que a execução do programa fique bloqueada durante a espera de um segundo.

Seja t_{atual} o tempo retornado pela função `millis()` e t_{anterior} o instante da última atualização do relógio.

A condição para incrementar os segundos será:

$$t_{\text{atual}} - t_{\text{anterior}} \geq 1000 \tag{1}$$

Quando essa condição for satisfeita, deverá ocorrer:

$$t_{\text{anterior}} = t_{\text{atual}} \tag{2}$$

$$segundos = segundos + 1 \tag{3}$$

A contagem deverá respeitar os limites convencionais de um relógio digital:

- Segundos: de 0 a 59.
- Minutos: de 0 a 59.
- Horas: de 0 a 23.

Quando os segundos atingirem 60, deverão retornar a zero, incrementando os minutos. Da mesma forma, quando os minutos atingirem 60, deverão retornar a zero, incrementando as horas. Quando as horas atingirem 24, deverão retornar a zero.

As regras de incremento podem ser representadas por:

$$segundos = \begin{cases} 0, & \text{se } segundos \geq 60 \\ segundos, & \text{caso contrário} \end{cases} \tag{4}$$

$$minutos = \begin{cases} minutos + 1, & \text{se } segundos = 0 \\ minutos, & \text{caso contrário} \end{cases} \tag{5}$$

$$horas = \begin{cases} 0, & \text{se } horas \geq 24 \\ horas, & \text{caso contrário} \end{cases} \tag{6}$$

O relógio deverá inicializar automaticamente com o horário $00:00:00$ sempre que o Arduino for ligado ou reiniciado.

3.2 Exibição das Informações no Display LCD

O display LCD deverá apresentar as informações de maneira organizada, permitindo que o usuário visualize o horário atual e identifique o modo de operação do sistema.

Na funcionalidade principal, a primeira linha deverá apresentar a identificação do relógio e a segunda linha deverá apresentar o horário.

Um exemplo de apresentação é:

```
RELOGIO DIGITAL
   00:00:00
```

Durante o funcionamento normal, o display deverá ser atualizado sempre que houver alteração no horário. Durante o modo de configuração, o display deverá pausar a contagem das horas e indicar qual campo está sendo ajustado. Um exemplo de apresentação é:

```
AJUSTE DE HORA
   00:00:00
```

Alternativamente, se quiser simular com realismo o ajuste do relógio, os dois dígitos do campo selecionado podem ficar piscando continuamente. A atualização do LCD não deverá utilizar atrasos bloqueantes que prejudiquem a leitura dos botões ou a contagem do tempo.

3.3 Funcionamento do Botão de Ajuste

O botão de ajuste deverá possuir duas funcionalidades, dependendo do tempo durante o qual permanecer pressionado.

O sistema deverá medir a duração de cada acionamento para distinguir um clique rápido de um pressionamento prolongado.

Seja $t_{\text{pressionado}}$ o instante em que o botão foi pressionado e t_{solto} o instante em que foi liberado.

O tempo de pressionamento será calculado por:

$$\Delta t = t_{\text{solto}} - t_{\text{pressionado}} \tag{7}$$

O comportamento deverá seguir as condições:

- Pressionamento prolongado: Quando o botão permanecer pressionado por 1000 ms ou mais, deverá ativar ou desativar o modo de configuração.

- Pressionamento rápido: Quando o botão for pressionado por menos de 1000 ms, E O MODO DE CONFIGURAÇÃO ESTIVER ATIVO, o sistema deverá alternar o campo que será ajustado.

O modo de configuração deverá funcionar da seguinte maneira:

1. Ao manter o botão pressionado, por pelo menos um segundo, o sistema deverá entrar no modo de ajuste.
2. Inicialmente, o campo selecionado será o dos segundos.
3. Cada clique rápido deverá alternar o campo selecionado, seguindo a sequência:

$$\text{Segundos} \rightarrow \text{Minutos} \rightarrow \text{Horas} \rightarrow \text{Segundos}$$

4. Ao manter o botão pressionado por pelo menos um segundo novamente, o sistema deverá sair do modo de configuração e retornar à operação normal.
5. Ao sair do modo de configuração, o relógio deverá continuar a contagem a partir do horário configurado.

Durante o modo de configuração, a contagem normal do relógio deverá permanecer pausada, evitando que os valores sejam alterados simultaneamente pelo temporizador e pelo usuário.

3.4 Funcionamento do Botão de Avanço

O botão de avanço será utilizado para incrementar o valor do campo selecionado durante o modo de configuração. Seu funcionamento deverá depender do campo atualmente selecionado.

- Ajuste das horas: Cada acionamento deverá incrementar uma unidade no valor das horas, respeitando o intervalo de 0 a 23.
- Ajuste dos minutos: Cada acionamento deverá incrementar uma unidade no valor dos minutos, respeitando o intervalo de 0 a 59.
- Ajuste dos segundos: Cada acionamento deverá incrementar uma unidade no valor dos segundos, respeitando o intervalo de 0 a 59.

Quando o limite máximo de cada campo for atingido, o próximo acionamento deverá retornar o valor para zero. Portanto, as operações deverão seguir:

$$horas = (horas + 1) \bmod 24 \tag{8}$$

$$minutos = (minutos + 1) \bmod 60 \tag{9}$$

$$segundos = (segundos + 1) \bmod 60 \tag{10}$$

O botão de avanço deverá ser ignorado quando o sistema não estiver no modo de configuração.

3.5 Opcional: Debounce

Os botões mecânicos podem apresentar oscilações elétricas durante o acionamento e a liberação, fenômeno conhecido como *bounce*. Essas oscilações podem provocar múltiplas detecções de um único pressionamento, comprometendo a seleção dos campos e a alteração dos valores. Portanto, deverá ser implementado um mecanismo de *debounce*, responsável por validar o estado do botão após um pequeno intervalo de estabilização. Recomenda-se utilizar um intervalo entre 20 ms e 50 ms para a validação do acionamento.

A implementação deverá permitir a identificação de:

- Transição de botão liberado para pressionado.
- Permanência do botão pressionado.
- Transição de botão pressionado para liberado.
- Duração total do pressionamento.

E o tratamento deverá ser realizado sem bloquear a execução principal do programa.

Um exemplo pode ser visto no link: `https://docs.arduino.cc/built-in-examples/digital/Debounce/`.

4 Requisitos do Projeto

1. Inicializar o relógio automaticamente no horário $00:00:00$ sempre que o Arduino for ligado.
2. Implementar uma base de tempo utilizando a função `millis()`, realizando a atualização dos segundos a cada 1000 ms.
3. Implementar a contagem de horas, minutos e segundos, respeitando os limites de 24 horas, 60 minutos e 60 segundos.
4. Exibir continuamente o horário no display LCD, utilizando o formato $HH:MM:SS$.
5. Implementar o botão de ajuste com dois tipos de acionamento:
   - Pressionamento prolongado, igual ou superior a 1 segundo, para entrar ou sair do modo de configuração.
   - Pressionamento rápido, inferior a 1 segundo, para alternar entre horas, minutos e segundos, se o modo de configuração estiver ativo.
6. Implementar o botão de avanço para incrementar o campo selecionado durante o modo de configuração.
7. Garantir que os valores de horas, minutos e segundos retornem a zero ao ultrapassar seus limites.
8. Ao sair do modo de configuração, o relógio deverá continuar a contagem a partir do horário configurado.

5 Observações

- A função `millis()` permite implementar tarefas periódicas sem interromper a execução principal do programa, diferentemente da função `delay()`, que bloqueia a execução durante determinado intervalo.
- Em um relógio comercial, seria recomendável utilizar um módulo RTC (*Real Time Clock*), como o DS3231, que possui oscilador próprio e bateria de reserva, permitindo manter a hora mesmo quando o microcontrolador estiver desligado.
- A estruturação em máquinas de estados permite organizar melhor o comportamento do sistema em diferentes modos de operação.